

Atividade extra - Unidade I - EDB 1

Aluna: Maria Mariana Varela Cavalcanti Souto

Matrícula: 20250033894

Turma: IMD0029 - ESTRUTURA DE DADOS BÁSICAS I - T03 (2026.1)

Link do repositório do github:

<https://github.com/marisout0/Atividade-extra-UNIDADE-1---EDB1---MARIA-MARIANA-SOUTO>

Questão 2.1

Pergunta: Menor valor de n para o qual $100n^2$ é mais rápido que 2^n .

Resolução: Precisamos encontrar o menor n tal que $100n^2 < 2^n$.

- Para $n = 14$: $100(14^2) = 19.600$ e $2^{14} = 16.384$ ($100n^2$ é mais lento).
- Para $n = 15$: $100(15^2) = 22.500$ e $2^{15} = 32.768$ ($100n^2$ é mais rápido).

Resposta: O menor valor é $n = 15$.

Questão 2.2

Pergunta: Para quais valores de n o método de trocas sucessivas ($8n^2$) é melhor que o de intercalação ($64n \log_2 n$)?

Resolução: Queremos $8n^2 < 64n \log_2 n$. Simplificando por $8n$: $n < 8 \log_2 n$.

- Para $n = 43$: $43 < 8 \log_2(43) \approx 8(5,42) = 43,36$ (Verdadeiro).
- Para $n = 44$: $44 < 8 \log_2(44) \approx 8(5,45) = 43,6$ (Falso).

Resposta: O método de trocas é melhor para $2 \leq n \leq 43$.

Questão 2.3

Pergunta: Expresse $f(n) = n^3/1000 - 100n^2 - 100n + 3$ na notação O .

Resolução: Na análise assintótica, focamos no termo de maior grau e ignoramos constantes e termos de ordem inferior.

Resposta: $O(n^3)$.

Questão 2.4

Pergunta: Maior tamanho n que pode ser resolvido em tempo t para diferentes complexidades (considerando $f(n)$ em microssegundos).

Exemplo (1 Segundo = $10^6 \mu s$):

- n^2 : $n^2 \leq 10^6 \Rightarrow n = 1.000$.
- 2^n : $2^n \leq 10^6 \Rightarrow n \approx 19$.

Resposta: Deve-se preencher a tabela calculando n para cada intervalo (minuto, hora, dia, etc.) igualando $f(n)$ ao tempo em microssegundos.

Questão 2.5

Pergunta: É verdade que $2^{n+1} = O(2^n)$? E que $2^{2n} = O(2^n)$?

Resolução:

1. $2^{n+1} = 2 \cdot 2^n$. Como 2 é uma constante, $2 \cdot 2^n \leq c \cdot 2^n$ é verdade para $c \geq 2$. **Sim**.
2. $2^{2n} = (2^n)^2$. Não existe constante c tal que $(2^n)^2 \leq c \cdot 2^n$ para todo n grande, pois 2^n cresce muito mais rápido que qualquer constante c . **Não**.

Questão 2.6

Pergunta: Quão mais lento o algoritmo fica se (i) duplicar n ou (ii) incrementar n em 1?

Para n^2 :

- (i) $(2n)^2 = 4n^2$ (**4 vezes mais lento**).
- (ii) $(n+1)^2 = n^2 + 2n + 1$.

Para 2^n :

- (i) 2^{2n} (**Quadraticamente mais lento**).
- (ii) $2^{n+1} = 2 \cdot 2^n$ (**2 vezes mais lento**).

Questão 2.7

Pergunta: Maior n para 1 hora de computação a 10^{10} op/seg ($t = 3,6 \cdot 10^{13}$ operações).

- (a) n^2 : $n = \sqrt{3,6 \cdot 10^{13}} \approx 6.000.000$.
- (e) 2^n : $2^n \leq 3,6 \cdot 10^{13} \Rightarrow n \approx 45$.

Questão 2.8

Pergunta: Reordene por taxa de crescimento.

Resposta: $f_2(n) = \sqrt{2}n < f_3(n) = n + 10 < f_1(n) = n^2 \cdot 5 < f_6(n) = n^2 \log_2 n < f_4(n) = 10^n < f_5(n) = 100^n$.

Questão 2.9 - Regra de Horner ($O(n)$)

Código em C++:

```
1 // Implementação  $O(n)$  usando a Regra de Horner
2 ✓ double horner(double coef[], int n, double x) {
3     double resultado = coef[n-1];
4     for (int i = n - 2; i >= 0; i--) {
5         resultado = resultado * x + coef[i];
6     }
7     return resultado;
8 }
```

Questão 2.10 - Contagem de Inversões ($O(n \log n)$)

Código em C++:

```
1 ✓ long long mergeAndCount(int arr[], int temp[], int left, int mid, int right) {
2     int i = left, j = mid, k = left;
3     long long count = 0;
4     while (i <= mid - 1 && j <= right) {
5         if (arr[i] <= arr[j]) temp[k++] = arr[i++];
6         else {
7             temp[k++] = arr[j++];
8             count += (mid - i); // Todos os elementos restantes na esquerda formam inversão
9         }
10    }
11    while (i <= mid - 1) temp[k++] = arr[i++];
12    while (j <= right) temp[k++] = arr[j++];
13    for (i = left; i <= right; i++) arr[i] = temp[i];
14    return count;
15 }
```